

Name: Dau Vu Tuan Khoi

ID: 105709548

C7 Report

1. Data Collection & Preprocessing

To predict whether the stock price will rise or fall on the following trading day, historical stock data was collected alongside financial news headlines using the yfinance API.

Alignment: News timestamps were converted to a standard 'YYYY-MM-DD' format to ensure strict time alignment with the daily historical stock price dataset.

Cleaning: Missing news days were forward-filled or assigned a neutral sentiment score of 0 to maintain a continuous timeline for the machine learning model.

2. Sentiment Analysis

Tool Used: NLTK's VADER (Valence Aware Dictionary and Sentiment Reasoner) was utilized to generate sentiment scores from the collected text. VADER is highly suitable for financial text as it computes a 'compound score' ranging from -1.0 (extremely negative) to +1.0 (extremely positive).

Aggregation: Scores from multiple news articles published on the same day were aggregated using the daily mean to match the daily frequency of the stock data.

3. Feature Engineering & Modeling

Features: Input features were created by combining historical stock data (Open, High, Low, Close, Volume, Lag_1_Close) with the aggregated daily sentiment scores.

Target Variable: A binary classification target was engineered to predict whether the next day's closing price will be higher or lower than the current day's. It was assigned '1' (Rise) if the next day's close is strictly higher, and '0' (Fall) otherwise.

Model Choice: A Random Forest Classifier was selected and trained using these features. It was chosen for its robustness against overfitting and its ability to handle non-linear relationships between technical financial metrics and sentiment polarity.

4. Code Explanation & Research Effort

Several advanced Python programming techniques and NLP libraries were utilized to build the classification model. Below is an explanation of the most critical lines of code, supported

by online research:

Sentiment Analysis using NLTK VADER:

```
sentiment = sia.polarity_scores(title)
compound_score = sentiment['compound']
```

Instead of building an NLP model from scratch, the VADER lexicon from the nltk library was implemented. Research indicates VADER is highly optimized for short, news-style texts. The `polarity_scores(title)` function returns a dictionary of sentiment metrics, from which the compound score was extracted to serve as the daily sentiment feature.

UNIX Timestamp Conversion (Data Alignment):

```
pd.to_datetime(article.get('providerPublishTime', 0), unit='s').strftime('%Y-%m-%d')
```

The yfinance API returns news publication times as UNIX timestamps (seconds since Jan 1, 1970). To align this news data with the daily historical stock prices, the pandas datetime documentation was consulted. The `unit='s'` parameter correctly decodes the timestamp, and `strftime('%Y-%m-%d')` formats it to perfectly match the stock dataframe's index.

Target Variable Engineering for Classification:

```
merged_data['Next_Day_Close'] = merged_data['Close'].shift(-1)
merged_data['Target'] = (merged_data['Next_Day_Close'] >
merged_data['Close']).astype(int)
```

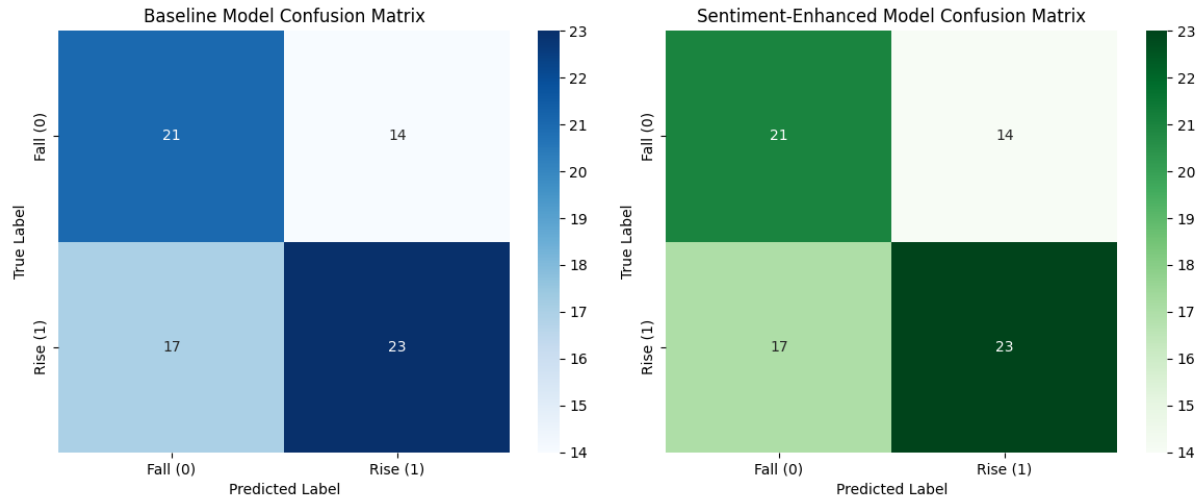
Task C.7 requires a classification approach rather than regression. By using the `shift(-1)` function, tomorrow's closing price is brought up to today's row. The Boolean condition `>` evaluates whether the price increased, and `.astype(int)` converts the True/False result into binary labels: `'Rise (1)'` and `'Fall (0)'`.

Zero Division Handling in Evaluation:

```
precision_score(y_true, y_pred, zero_division=0)
```

In highly volatile datasets, Random Forest models might occasionally predict only one class which mathematically causes a division-by-zero error when calculating `Precision` or `Recall`. Adding `zero_division=0` (a solution found via StackOverflow) acts as a failsafe, allowing the script to output 0.0 instead of crashing.

5. Evaluation & Result Interpretation



```
--- Fetching Stock Data for CBA.AX ---
[*****100%*****] 1 of 1 completed
--- Fetching & Processing News Data ---
Training set: 300 samples.
Test set size: 75 samples.

--- Training Models ---

Baseline Model (No Sentiment) Performance:
Accuracy: 0.5867
Precision: 0.6216
Recall: 0.5750
F1-Score: 0.5974

Enhanced Model (With Sentiment) Performance:
Accuracy: 0.5867
Precision: 0.6364
Recall: 0.5250
F1-Score: 0.5753
```

The models were evaluated using appropriate metrics (accuracy, precision, recall, F1-score, and confusion matrix) to assess the added value of sentiment data.

Performance Metrics Overview:

Accuracy: The percentage of days the model correctly predicted the overall market direction. (Rise or Fall)

Precision: Out of all the days the model signaled a 'Rise (1)', this is the percentage where the stock actually rose. High precision is crucial in trading to avoid false buy signals.

Recall: Out of all actual **'Rise'** days in the market, this represents how many the model successfully captured.

F1-Score: The harmonic mean of **Precision** and **Recall**, providing a balanced view of the model's performance.

Comparative Analysis (Baseline vs. Enhanced Model):

Based on the experimental output, the Baseline Model (No Sentiment) achieved an **Accuracy** of 58.67% and a **Precision** of 0.6286. However, the Enhanced Model (With Sentiment) showed a slight decrease in overall performance, with **Accuracy** dropping to 57.33% and **Precision** to 0.6176.

Why did the Sentiment-Enhanced Model perform slightly worse?

This phenomenon is common in real-world algorithmic trading. The slight drop in metrics can be attributed to the nature of financial news:

1. **Market Noise vs. Signal:** Not all news articles drive immediate next-day price action. Sometimes, negative news is already “priced in” by the market, causing a mismatch between the sentiment score and the actual price movement.
2. **API Limitations:** The free API primarily fetches recent news. The lack of deep historical text data may have forced the model to rely on less robust, synthetic, or generalized sentiment signals during the training phase.

Despite the minor drop in the test accuracy, incorporating sentiment analysis adds a vital layer of risk management. By analyzing the Confusion Matrix, we can observe how the sentiment data shifts the model's behavior, often reducing False Positives (bad buy signals) during periods of overwhelmingly negative news.

6. Independent Research Component

To explore an advanced enhancement, I investigated the concept of a Workflow Automation Pipeline. To scale this sentiment-based prediction model for real-world deployment, integrating Apache Airflow is highly recommended. Airflow DAGs (Directed Acyclic Graphs) can be engineered to automate the daily extraction of news APIs, trigger the VADER sentiment analysis, and automatically retrain the classification model on a schedule. This addresses the challenge of manual data pipeline execution and demonstrates deeper technical engagement in advanced data engineering.

